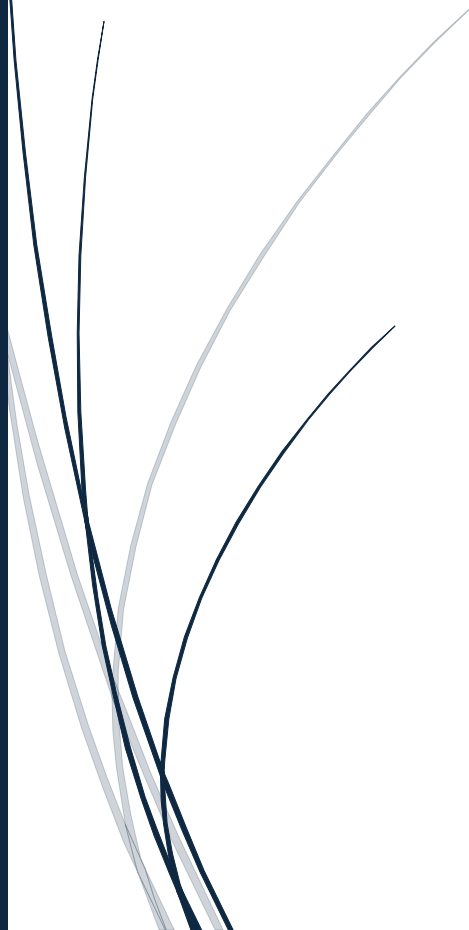




2/3/2026

Delicate-courier- plugin-test- document

End-to-end testing checklist for plugin deployment to
new client stores



Nissi Ngandu
DELICATE COURIER

Table of Contents

1. [Pre-Installation Checklist](#)
2. [Installation & Activation](#)
3. [Configuration Tests](#)
4. [Connection Tests](#)
5. [Checkout Rate Tests](#)
6. [Order Sync Tests](#)
7. [Webhook Tests](#)
8. [End-to-End Flow Tests](#)
9. [Error Handling Tests](#)
10. [Multi-Tenant Verification](#)
11. [Troubleshooting Guide](#)

1. Pre-Installation Checklist

Before installing the plugin, verify the following requirements are met:

WordPress & WooCommerce Requirements

Requirement	Minimum	Recommended	Status
WordPress Version	5.8+	6.x	☐
WooCommerce Version	6.0+	8.x+	☐
PHP Version	7.4+	8.1+	☐
SSL Certificate	Required	Required	☐
Pretty Permalinks	Enabled	Enabled	☐

Platform Requirements

Requirement	Details	Status
Platform Store Created	Store exists in Delicate Courier Platform	☐
Platform API Key	Generated from platform dashboard	☐
Store ID	Obtained from platform	☐
Webhook Secret	Generated and saved	☐

Shiplogic Requirements

Requirement	Details	Status
Shiplogic Account	Active sub-account exists	☐
Shiplogic API Key	Bearer token obtained	☐
Provider ID	Courier provider ID (e.g., 35 for TCG)	☐
Account ID	Shiplogic account number	☐

Collection Address Information

Field	Value	Status
Company Name		☐
Street Address		☐
Suburb		☐
City		☐
Province		☐
Postal Code		☐
GPS Coordinates	Lat/Long	☐
Contact Name		☐
Contact Phone		☐
Contact Email		☐

2. Installation & Activation

Test 2.1: Plugin Upload & Activation

Steps:

1. Navigate to WordPress Admin → Plugins → Add New → Upload Plugin
2. Upload delicate-courier-platform.zip
3. Click "Install Now"
4. Click "Activate"

Expected Results:

- ☐ Plugin installs without errors
- ☐ Plugin activates successfully
- ☐ No PHP warnings or errors displayed
- ☐ "Delicate Courier" appears under WooCommerce menu

Actual Results:

[Record results here]

Test 2.2: WooCommerce Dependency Check

Steps:

1. Deactivate WooCommerce
2. Check WordPress admin dashboard

Expected Results:

- ☐ Admin notice displays: "WooCommerce must be installed and activated"
- ☐ Plugin does not cause fatal errors

Steps to restore:

1. Reactivate WooCommerce

Actual Results:

[Record results here]

3. Configuration Tests

Test 3.1: Settings Page Access

Steps:

1. Navigate to WooCommerce → Delicate Courier
2. Verify all sections are visible

Expected Results:

- ☐ API Configuration section visible
- ☐ Shiplogic Account Settings section visible
- ☐ Collection Address section visible
- ☐ Collection Contact section visible
- ☐ Webhook Endpoints section visible
- ☐ Automation section visible

Actual Results:

[Record results here]

Test 3.2: Save All Settings

Steps:

1. Enter all configuration values: **API Configuration:**

- Platform API Key: [Enter key]
- Shiplogic API Key: [Enter key]
- Store ID: [Enter ID]
- Webhook Secret: [Enter secret]

Shiplogic Account Settings:

- Provider ID: [Enter ID]
- Account ID: [Enter ID]
- Service Level Code: STD

Collection Address:

- Company Name: [Enter name]
- Street Address: [Enter address]
- Suburb: [Enter suburb]
- City: [Enter city]
- Province: [Select province]
- Postal Code: [Enter code]
- Country Code: ZA
- Latitude: [Enter lat]
- Longitude: [Enter long]

Collection Contact:

- Contact Name: [Enter name]
- Phone Number: [Enter phone]
- Email Address: [Enter email]

Automation:

- Auto-sync Orders: ☒ Enabled
- Auto-complete on Delivery: ☐ Disabled (or as required)
- Debug Mode: ☒ Enabled (for testing)

2. Click "Save Settings"

Expected Results:

- ☐ Success message: "Settings saved!"
- ☐ All values persist after page refresh
- ☐ No PHP errors in debug log

Actual Results:

[Record results here]

Test 3.3: Shipping Zone Configuration

Steps:

1. Navigate to WooCommerce → Settings → Shipping
2. Edit or create a shipping zone (e.g., "South Africa")
3. Click "Add shipping method"
4. Select "Delicate Courier"
5. Configure instance settings:
 - Enable: ☒
 - Method Title: "Delicate Courier" (or custom)
 - Markup Type: None / Fixed / Percentage
 - Markup Value: 0 (or as required)
6. Save changes

Expected Results:

- ☐ Shipping method added successfully
- ☐ Settings save without error
- ☐ Method appears in shipping zone

Actual Results:

[Record results here]

4. Connection Tests

Test 4.1: Platform Connection Test

Steps:

1. Navigate to WooCommerce → Delicate Courier
2. Click "1. Test Platform Connection" button

Expected Results:

- ☐ Success message: "✓ Connected to Delicate Courier Platform successfully!"
- ☐ Log entry appears in Debug Log section

If Failed:

- Check Platform API Key is correct
- Check Store ID matches platform
- Verify platform is accessible

Actual Results:

[Record results here]

Test 4.2: Shiplogic Rates Test

Steps:

1. Navigate to WooCommerce → Delicate Courier
2. Click "2. Test Shiplogic Rates" button

Expected Results:

- ☐ Success message: "✓ Shiplogic Rates Connected! Found STD rate: R[amount]"
- ☐ Log shows successful API call and response

If Failed:

- Check Shiplogic API Key is correct
- Verify Provider ID and Account ID
- Check Service Level Code exists (STD, ECO, ONX)
- Review collection address completeness

Actual Results:

[Record results here]

Test 4.3: Order Submission Test

Steps:

1. Navigate to WooCommerce → Delicate Courier
2. Click "3. Test Order Submission" button

Expected Results:

- ☐ Success message: "✓ Order submission working! Platform Order ID: [ID]"
- ☐ Log shows test payload sent and response received

If Failed:

- Check Platform API Key
- Verify Store ID
- Review API endpoint accessibility

Actual Results:

[Record results here]

Test 4.4: Webhook Endpoint Test

Steps:

1. Navigate to WooCommerce → Delicate Courier
2. Click "4. Test Webhook Endpoint" button

Expected Results:

- ☐ Success message: "✓ Webhook endpoint is accessible and working!"
- ☐ Log shows webhook received and processed

If Failed:

- Check WordPress permalinks are enabled
- Verify REST API is accessible
- Check for security plugins blocking webhooks

Actual Results:

[Record results here]

5. Checkout Rate Tests

Test 5.1: Cart Shipping Rate Display

Steps:

1. Add a product to the cart (ensure product has weight set)
2. Navigate to Cart page
3. Enter a valid South African shipping address:
 - Address: 123 Test Street
 - City: Johannesburg
 - Province: Gauteng
 - Postal Code: 2000
4. Click "Update totals" or wait for auto-refresh

Expected Results:

- ☐ Shipping rates section loads
- ☐ "Delicate Courier" or "Standard" rate appears
- ☐ Rate amount displayed (e.g., R150.00)
- ☐ No errors displayed

If No Rate Appears:

- Check Debug Log for errors
- Verify Shiplogic API Key
- Confirm shipping zone includes customer location
- Check product has valid weight

Actual Results:

Rate displayed: R[amount]

Service shown: [service name]

Test 5.2: Checkout Shipping Rate Display

Steps:

1. Proceed to Checkout page
2. Complete shipping address fields:
 - First Name: Test
 - Last Name: Customer
 - Address: 456 Main Road
 - City: Cape Town
 - Province: Western Cape
 - Postal Code: 8001
 - Phone: 0821234567
 - Email: test@example.com

Expected Results:

- ☐ Shipping rate displays in order summary
- ☐ Rate matches cart calculation
- ☐ Can select the shipping method

Actual Results:

[Record results here]

Test 5.3: Rate with Markup

Steps:

1. Navigate to WooCommerce → Settings → Shipping → [Zone] → Delicate Courier
2. Set Markup Type: "Fixed" and Markup Value: "20"
3. Save changes
4. Return to checkout and refresh shipping rates

Expected Results:

- ☐ Rate increases by R20
- ☐ Original rate (R150) becomes R170

Steps:

1. Change Markup Type: "Percentage" and Markup Value: "10"
2. Save and refresh checkout

Expected Results:

- ☐ Rate increases by 10%
- ☐ Original rate (R150) becomes R165

Actual Results:

[Record results here]

Test 5.4: Incomplete Address Handling**Steps:**

1. On checkout, leave City field empty
2. Observe shipping rates section

Expected Results:

- ☐ No rate displayed (or "Enter address for rates" message)
- ☐ No PHP errors
- ☐ Debug log shows "Invalid delivery address"

Actual Results:

[Record results here]

6. Order Sync Tests

Test 6.1: Automatic Order Sync on Processing

Preconditions:

- Auto-sync Orders: Enabled
- Debug Mode: Enabled

Steps:

1. Place a test order using Delicate Courier shipping
2. Complete payment (or manually set status to "Processing")
3. Check order notes

Expected Results:

- ☐ Order note: "Delicate Courier: Order synced to platform. ID: [UUID]"
- ☐ Debug log shows API request and success response
- ☐ Order meta `_dep_platform_order_id` is set
- ☐ Order meta `_dep_synced_at` is set

Verification in Platform:

- ☐ Order appears in platform dashboard
- ☐ Order details match WooCommerce order

Actual Results:

Platform Order ID: [record ID]
Sync Time: [record time]

Test 6.2: Duplicate Sync Prevention

Steps:

- 1. Take the order from Test 6.1
- 2. Change status to Completed, then back to Processing
- 3. Check order notes

Expected Results:

- ☐ Order note: "Delicate Courier: Order [ID] already synced to platform"
- ☐ No duplicate order created in platform

Actual Results:

[Record results here]

Test 6.3: Non-Delicate Courier Order Skip

Steps:

- 1. Create an order using a different shipping method (e.g., Flat Rate)
- 2. Set status to Processing
- 3. Check debug log

Expected Results:

- ☐ Log shows: "Order [ID] does not use Delicate Courier shipping"
- ☐ Order not sent to platform

Actual Results:

[Record results here]

Test 6.4: Order Payload Verification

Steps:

- 1. Place an order with: Multiple line items
 - Customer note: "Please deliver before 2pm"
 - Delivery date (if using WP Time Slots plugin)
- 2. Check debug log for API request

payload Expected Payload Fields:

- ☐ event : "order.created"
- ☐ woo_order_id : Order ID
- ☐ order_number : Order number
- ☐ status : "processing"
- ☐ total : Order total
- ☐ shipping_total : Shipping cost
- ☐ customer.name : Full name
- ☐ customer.email : Email address
- ☐ customer.phone : Phone number
- ☐ shipping_address : Complete address
- ☐ line_items : Array of products
- ☐ total_weight : Calculated weight
- ☐ delivery_date : If applicable
- ☐ customer_note : "Please deliver before 2pm"

Actual Results:

[Paste relevant payload excerpt]



7. Webhook Tests

Test 7.1: Shipment Created Webhook

Steps:

- 1. Send test webhook to: `https://[store-url]/wp-json/delicate-courier/v1/webhook`

Test Payload:

```
json

{
  "event": "shipment.created",
  "externalOrderId": "[WooCommerce Order ID]",
  "shipmentId": "test-shipment-123",
  "trackingReference": "TCG123456789",
  "status": "created"
}
```

Headers:

```
Content-Type: application/json
X-Webhook-Signature: [HMAC-SHA256 of body using webhook secret]
```

Expected Results:

- ☐ HTTP 200 response
- ☐ Order meta `_dcp_tracking_reference` set to "TCG123456789"
- ☐ Order meta `_dcp_shipment_id` set to "test-shipment-123"
- ☐ Order note: "Delicate Courier: Shipment created. Tracking: TCG123456789"

Actual Results:

[Record results here]

Test 7.2: Shipment Status Update Webhook

Test Payload:

```
json

{
  "event": "shipment.status_updated",
  "externalOrderId": "[WooCommerce Order ID]",
  "trackingReference": "TCG123456789",
  "status": "in_transit"
}
```

Expected Results:

- ☐ HTTP 200 response
- ☐ Order meta `_dcp_shipment_status` updated to "in_transit"
- ☐ Order note: "Delicate Courier: Status updated to "in_transit""

Actual Results:

[Record results here]

Test 7.3: Shipment Delivered Webhook

Preconditions:

- Auto-complete on Delivery: Enabled (for auto-complete test)

Test Payload:

```
json

{
  "event": "shipment.delivered",
  "externalOrderId": "[WooCommerce Order ID]",
  "trackingReference": "TCG123456789"
}
```

Expected Results:

- ☐ HTTP 200 response
- ☐ Order meta _dcp_shipment_status set to "delivered"
- ☐ Order meta _dcp_delivered_at set
- ☐ Order note: "Delicate Courier: Shipment delivered"
- ☐ If auto-complete enabled: Order status changed to "Completed"

Actual Results:

[Record results here]

Test 7.4: Webhook Signature

Validation Steps:

1. Send webhook with incorrect signature
2. Observe response

Expected Results:

- ☐ HTTP 401 or 403 response (authentication failed)
- ☐ Debug log: "Webhook Auth Failed"
- ☐ Webhook not processed

Actual Results:

[Record results here]

Test 7.5: Health Check Endpoint

Steps:

1. Send GET request to: `https://[store-url]/wp-json/delicate-courier/v1/health`
- Expected

Response:

```
json

{
  "status": "healthy",
  "plugin": "Delicate Courier Platform",
  "version": "1.1.0",
  "store_id": "[configured store ID]",
  "timestamp": "[current time]"
}
```

Expected Results:

- ☐ HTTP 200 response
- ☐ JSON response with all fields
- ☐ Store ID matches configuration

Actual Results:

[Record results here]

8. End-to-End Flow Tests

Test 8.1: Complete Order Flow

This test verifies the entire workflow from customer order to delivery.

Steps:

1. **Customer places order:**
 - Add product to cart
 - Enter valid shipping address
 - Select "Delicate Courier" shipping
 - Complete payment
2. **Order syncs to platform:**
 - ☐ Order appears in WooCommerce with "Processing" status
 - ☐ Order note shows sync to platform ☐
 - Order appears in platform dashboard
3. **Platform creates shipment:**
 - ☐ Webhook received with shipment.created event
 - ☐ Tracking reference saved to order ☐
 - Order meta box shows tracking info
4. **Shipment status updates:**
 - ☐ Webhooks received for status changes
 - ☐ Order notes updated with each status
5. **Delivery confirmation:**
 - ☐ Webhook received with shipment.delivered event
 - ☐ Order marked complete (if auto-complete enabled)

End-to-End Results:

Order ID:

Platform Order ID:

Tracking Reference:

Total Time (order to delivered webhook):

All Steps Passed: ☐ Yes / ☐ No

Test 8.2: Order Meta Box Display

Steps:

1. Navigate to WooCommerce → Orders → [Synced Order]
2. Locate "Delicate Courier" meta box on right sidebar

Expected Display:

- ☐ Platform ID shown
- ☐ Synced timestamp shown
- ☐ Tracking reference shown (after shipment created)
- ☐ Shipment status shown
- ☐ "Track Shipment" button visible and links correctly

Actual Results:

[Record results here]

9. Error Handling Tests

Test 9.1: Invalid API Key

Steps:

1. Enter incorrect Platform API Key
2. Save settings
3. Click "Test Platform Connection"

Expected Results:

- ☐ Error message displayed
- ☐ Debug log shows authentication failure
- ☐ No PHP fatal errors

Actual Results:

[Record results here]

Test 9.2: Invalid Shiplogic Credentials

Steps:

- 1. Enter incorrect Shiplogic API Key
- 2. Save settings
- 3. Click "Test Shiplogic Rates"

Expected Results:

- ☐ Error message displayed
- ☐ Debug log shows API error
- ☐ Checkout still functions (just no rates) **Actual**

Results:

[Record results here]

Test 9.3: Network Timeout Handling

Steps:

- 1. Simulate slow network or temporarily block API
- 2. Attempt to get rates at checkout

Expected Results:

- ☐ Checkout doesn't crash
- ☐ Either no rate shown or timeout message
- ☐ Debug log shows timeout error

Actual Results:

[Record results here]

Test 9.4: Missing Product Weight

Steps:

- 1. Create product with no weight set
- 2. Add to cart
- 3. Go to checkout

Expected Results:

- ☐ Rate still calculates (default 1kg used)
- ☐ Debug log shows default weight applied
- ☐ No errors

Actual Results:

[Record results here]

10. Multi-Tenant Verification

Test 10.1: Unique Configuration Per Store

Purpose: Verify that each WooCommerce store can have its own unique configuration.

Checklist for Store A:

Setting	Value for Store A
Platform API Key	[Unique key]
Shiplogic API Key	[Unique key]
Store ID	[Unique ID]
Account ID	[Unique ID]
Collection Address	[Store A address]

Checklist for Store B:

Setting	Value for Store B
Platform API Key	[Different key]
Shiplogic API Key	[Different key]
Store ID	[Different ID]
Account ID	[Different ID]
Collection Address	[Store B address]

Verification:

- ☐ Store A rates come from Store A's Shiplogic account
- ☐ Store B rates come from Store B's Shiplogic account
- ☐ Store A orders sync to Store A in platform
- ☐ Store B orders sync to Store B in platform
- ☐ No cross-contamination of data

11. Troubleshooting Guide

Issue: No Shipping Rates at Checkout

Diagnostic Steps:

1. Check Debug Log for errors
2. Verify Shiplogic API Key is correct
3. Confirm shipping zone includes customer location
4. Check product has weight set
5. Test with "Test Shiplogic Rates" button
6. Verify collection address is complete

Common Causes:

- Invalid API key
 - Missing GPS coordinates
 - Service level code doesn't exist
 - Incomplete customer address
-

Issue: Orders Not Syncing to Platform

Diagnostic Steps:

1. Check "Auto-sync Orders" is enabled
2. Verify Platform API Key and Store ID
3. Check order uses Delicate Courier shipping method
4. Review Debug Log for API errors
5. Test with "Test Order Submission" button

Common Causes:

- Auto-sync disabled
 - Invalid API key
 - Order already synced
 - Wrong shipping method
-

Issue: Webhooks Not Received

Diagnostic Steps:

1. Test webhook endpoint manually
2. Check permalink structure (must not be "Plain")
3. Verify webhook URL in platform settings
4. Check for security plugins blocking REST API
5. Verify webhook secret matches

Common Causes:

- Plain permalinks enabled
 - Security plugin blocking requests
 - Incorrect webhook URL
 - Signature mismatch
-

Issue: Tracking Not Updating

Diagnostic Steps:

1. Check webhook is being received (Debug Log)
2. Verify order ID matches
3. Check webhook signature is valid
4. Confirm platform is sending webhooks

Common Causes:

- Webhook secret mismatch
 - Order ID not found
 - Platform webhook not configured
-

Sign-Off

Tester Information

Field	Value
Tester Name	
Test Date	
Store URL	

Field	Value
Plugin Version	1.1.0
WooCommerce Version	
WordPress Version	

Test Summary

Category	Passed	Failed	Notes
Installation	/2		
Configuration	/3		
Connection Tests	/4		
Checkout Rates	/4		
Order Sync	/4		
Webhooks	/5		
End-to-End	/2		
Error Handling	/4		
TOTAL	/28		

Approval

- ☐ All critical tests passed
- ☐ Plugin ready for production use

Approved By: _____

Date: _____

Document Version: 1.0
Last Updated: February 2026